

Sistema Bancario .NET

Arquitectura de Acceso a Datos Híbrida:

ADO.NET & Entity Framework Core

Abiud Vera

1. Modelado de Datos

Definición del núcleo del negocio mediante entidades fuertemente tipadas y configuración del contexto de base de datos.

Entidades Principales



Cliente

Almacena la identidad del usuario (Documento único, Email y Nombre Completo). Mantiene una relación de uno a muchos con sus respectivas Cuentas.



Cuenta

Registra el saldo y el estado activo/inactivo. Vinculada a un Clienteld y con una colección interna de Movimientos.



Movimiento

Auditoría inmutable de transacciones. Registra Monto, Fecha, Concepto y Tipo (enumerador estricto para Depósitos, Retiros y Transferencias).

Contexto de Base de Datos

- > **Fluent API:** Creación de reglas estrictas en `OnModelCreating` sobrescribiendo el modelo por defecto.
- > **Integridad de Datos:** Índices únicos aplicados sobre Documento (Cliente) y NumeroCuenta (Cuenta) para evitar duplicidades.
- > **Mapeo Preciso:** Configuración de precisión decimal (18, 2) vital para el correcto manejo de saldos monetarios.
- > **Comportamiento en Cascada:** Configuración de `onDelete(DeleteBehavior.Cascade)` asegurando la limpieza automática de registros dependientes.

```
7 public class BancoContext : DbContext
8 {
9     1 reference
10    public DbSet<Cliente> Clientes { get; set; } = null!;
11    8 references
12    public DbSet<Cuenta> Cuentas { get; set; } = null!;
13    4 references
14    public DbSet<Movimiento> Movimientos { get; set; } = null!;
15
16    0 references
17    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
18    {
19        optionsBuilder.UseSqlServer(DatabaseHelper.ConnectionString);
20    }
21
22    0 references
23    protected override void OnModelCreating(ModelBuilder modelBuilder)
24    {
25        modelBuilder.Entity<Cliente>()
26            .HasIndex(x => x.Documento)
27            .IsUnique();
28
29        modelBuilder.Entity<Cuenta>()
30            .HasIndex(x => x.NumeroCuenta)
31            .IsUnique();
32
33        modelBuilder.Entity<Cuenta>()
34            .Property(x => x.Saldo)
35            .HasPrecision(18, 2);
36    }
37 }
```

2. Acceso a Datos con ADO.NET

Implementación de servicios de alto rendimiento mediante la ejecución directa de instrucciones SQL contra SQL Server.

AdoNetService: Gestión de Clientes

- **Control Total SQL:** Sentencias SQL puras ejecutadas a través de SqlCommand.
- **Prevención de Inyecciones:** Uso riguroso de colecciones Parameters.AddWithValue para asegurar todos los inputs del sistema.
- **Gestión de Conexiones:** Implementación del patrón using en SqlConnection asegurando la liberación temprana de recursos.
- **Lectura Eficiente:** Uso de SqlDataReader para iterar sobre los resultados sin sobrecargar la memoria en operaciones de listado masivo.

```
6 public class AdoNetService
7 {
8     2 references
9     public void RegistrarCliente(string nombreCompleto, string documento, string email)
10     {
11         const string sql = @"
12             INSERT INTO Clientes (NombreCompleto, Documento, Email)
13             VALUES (@NombreCompleto, @Documento, @Email)";
14
15         using var connection = new SqlConnection(DatabaseHelper.ConnectionString);
16         using var command = new SqlCommand(sql, connection);
17         command.Parameters.AddWithValue("@NombreCompleto", nombreCompleto);
18         command.Parameters.AddWithValue("@Documento", documento);
19         command.Parameters.AddWithValue("@Email", email);
20
21         connection.Open();
22         var filas = command.ExecuteNonQuery();
23         Console.WriteLine($"Cliente registrado con éxito ({filas} fila(s) afectada(s)).");
24     }
25
26     2 references
27     public void ListarClientes()
28     {
29         const string sql = "SELECT Id, NombreCompleto, Documento, Email FROM Clientes ORDER BY Id";
30
31         using var connection = new SqlConnection(DatabaseHelper.ConnectionString);
32         using var command = new SqlCommand(sql, connection);
```

3. Acceso a Datos con EF Core

Mapeo Objeto-Relacional para la lógica de negocio compleja,
transacciones de múltiples pasos y abstracción de consultas
estructuradas.

EfCoreService: Registro y Consultas

- **Operaciones Conjuntas:** El método RegistrarCuenta crea la cuenta bancaria y el movimiento inicial en un único evento en memoria antes de guardar.
- **Proyecciones DTO:** Uso inteligente de .Select(c => new CuentaDTO) para retornar únicamente la data requerida a la capa de presentación.
- **Rendimiento:** Implementación de .AsNoTracking() en listados y búsquedas de solo-lectura, aligerando la carga en el procesador EF Core.
- **LINQ Dinámico:** Búsquedas textuales aplicadas directamente sobre las propiedades de navegación cruzada.

```
8 public class EfCoreService
9 {
10     2 references
11     public void RegistrarCuenta(int clienteId, string numeroCuenta, decimal saldoInicial)
12     {
13         using var context = new BancoContext();
14
15         var clienteExiste = context.Clientes.Any(x => x.Id == clienteId);
16         if (!clienteExiste)
17         {
18             Console.WriteLine($"No existe el cliente {clienteId}, no se pudo registrar la cuenta.");
19             return;
20         }
21
22         var cuenta = new Cuenta
23         {
24             ClienteId = clienteId,
25             NumeroCuenta = numeroCuenta,
26             Saldo = saldoInicial,
27             Activa = true
28         };
29
30         context.Cuentas.Add(cuenta);
31
32         context.Movimientos.Add(new Movimiento
33         {
34             Cuenta = cuenta,
35             Fecha = DateTime.Now,
```

Transacciones Seguras (ACID)

- **Atomicidad:** Implementación estricta mediante `BeginTransaction()` envolviendo el proceso completo de transferencia.
- **Reglas de Negocio:** Múltiples bloques condicionales aseguran que las cuentas existan, estén activas y posean los fondos necesarios antes de operar.
- **Consistencia:** Descuento en cuenta origen, abono en cuenta destino y registro doble en la tabla Movimientos en un mismo bloque lógico.
- **Rollback de Seguridad:** Uso de bloques try-catch para revertir cualquier cambio incompleto (Rollback) garantizando que nunca exista pérdida de fondos.

```
public void RealizarTransferencia(int cuentaOrigenId, int cuentaDestinoId, decimal monto, string concepto)
{
    using var context = new BancoContext();
    using var transaccion = context.Database.BeginTransaction();

    try
    {
        var cuentaOrigen = context.Cuentas.FirstOrDefault(c => c.Id == cuentaOrigenId);
        var cuentaDestino = context.Cuentas.FirstOrDefault(c => c.Id == cuentaDestinoId);

        if (cuentaOrigen is null || cuentaDestino is null)
        {
            throw new InvalidOperationException("Alguna de las cuentas no existe.");
        }

        if (!cuentaOrigen.Activa || !cuentaDestino.Activa)
        {
            throw new InvalidOperationException("Ambas cuentas deben estar activas.");
        }

        if (cuentaOrigen.Saldo < monto)
        {
            throw new InvalidOperationException("Saldo insuficiente en la cuenta de origen.");
        }

        cuentaOrigen.Saldo -= monto;
        cuentaDestino.Saldo += monto;
```

Comparativa Tecnológica del Proyecto

El enfoque ADO.NET

Implementado en este proyecto para la gestión de clientes, demostrando un paradigma de bajo nivel. Brinda un **control absoluto sobre la ejecución de consultas**, minimiza la sobrecarga de memoria, y es ideal para rutinas y procedimientos en lote altamente optimizados donde el mapeo automático no es estrictamente necesario.

El ecosistema EF Core

Seleccionado para manejar las cuentas y transferencias debido a su poder para mantener consistencia transaccional y navegar entidades relacionadas. Aumenta drásticamente la **velocidad de desarrollo**, **reduce la cantidad de código repetitivo** y permite manipular el estado de los datos usando conceptos completamente orientados a objetos.

Gracias por su atención.